Output Video – Implementation / Pipeline Development

1.Objective and Purpose of the System / Workflow

The objective of this system is to design and implement a scalable, efficient, and automated ETL (Extract, Transform, Load) pipeline using dbt (Data Build Tool) and Snowflake within the domain of data engineering and big data. The system aims to transform raw and unstructured data into clean, structured, and analytics-ready datasets that can support business intelligence and data-driven decision-making.

The purpose of this workflow is to streamline the data processing lifecycle by automating data transformations, improving data quality, and ensuring consistency across datasets. By leveraging dbt for modular data modeling and Snowflake for cloud-based storage and processing, the system provides a reliable and high-performance data pipeline. Additionally, the workflow supports real-time or near real-time data availability and integrates with a web application layer to deliver meaningful insights through visualization.

Overall, the system enhances efficiency, reduces manual intervention, and provides a robust foundation for scalable data analytics in modern organizations.

2.Tools and Technologies Used

The implementation of the **dbt + Snowflake ETL pipeline** utilizes a combination of modern data engineering tools and technologies to ensure efficient data processing, transformation, and visualization.

Category	Tool / Technology	Purpose
Data Transformation	dbt (Data Build Tool)	Used for transforming and modeling data using SQL-based workflows
Data Warehouse	Snowflake	Cloud-based platform for storing and querying large-scale data
Programming Language	SQL	Used to write transformation queries in dbt models
	Python	Used for automation, scripting, and integration tasks
Cloud Platform	AWS / Azure / GCP	Provides infrastructure for deployment and scalability
Web Framework	Streamlit / Flask	Used to build web-based dashboards and visualization interfaces
Version Control	Git / GitHub	Used for code management and collaboration
Orchestration (Optional)	Apache Airflow	Used for scheduling and automating workflow execution

A. dbt (Data Build Tool)

dbt is the core tool used for data transformation and modeling within the pipeline. It allows developers to write modular SQL queries, known as models, which transform raw data into structured formats. dbt also supports version control, documentation, and testing, enabling better maintainability and reliability of the data pipeline. Additionally, dbt provides features such as incremental models and dependency management, which improve performance and scalability.

B. Snowflake (Cloud Data Warehouse)

Snowflake serves as the primary data storage and processing platform. It is a cloud-based data warehouse that supports large-scale data storage, fast query execution, and automatic scaling. Snowflake separates storage and compute resources, allowing efficient resource management. It also supports structured and semi-structured data, making it suitable for big data applications.

C. SQL (Structured Query Language)

SQL is used extensively in this project to define transformation logic within dbt models. It is used for data cleaning, filtering, joining multiple datasets, and performing aggregations. SQL enables efficient manipulation of large datasets stored in Snowflake and is essential for implementing business logic.

D. Python

Python is used for automation, data integration, and scripting tasks within the pipeline. It helps in tasks such as data ingestion from external sources, preprocessing, and triggering dbt workflows. Python libraries can also be used for additional data processing and integration with other services.

E. Cloud Platform (AWS / Azure / GCP)

A cloud platform is used to host the entire system, providing scalability, flexibility, and high availability. It supports data storage, compute resources, and deployment of applications. Cloud infrastructure ensures that the system can handle large volumes of data and scale according to demand.

F. Streamlit / Flask (Web Application Framework)

These frameworks are used to develop the web-based dashboard for data visualization. The web application provides an interactive interface for users to view reports, charts, and insights generated from the processed data. This enhances usability and supports better decision-making.

G. Git / GitHub (Version Control System)

Git and GitHub are used for managing the project code and maintaining version history. They enable collaboration, tracking of changes, and rollback of previous versions if needed. This ensures proper project management and code reliability.

H. Apache Airflow (Optional – Workflow Orchestration)

Apache Airflow can be used to schedule and automate the execution of the ETL pipeline. It helps in managing dependencies between tasks and ensures that workflows are executed in the correct order. Airflow also provides monitoring and logging features for better pipeline management.

3.System Setup (Databases, Schemas, Tables, Configuration of Resources)

A. Overview of System Setup

The system setup defines the configuration and organization of the data pipeline components required for implementing the dbt + Snowflake ETL workflow. It includes the creation of databases, schemas, tables, and the configuration of computational resources to ensure efficient data processing, transformation, and storage.

B. Database Setup (Snowflake Environment)

The system uses Snowflake as the primary data warehouse. A dedicated database is created to manage all project-related data.

- A database (e.g., **DATA_PIPELINE_DB**) is created to store all datasets.
- Snowflake provides a centralized environment for managing structured and semi-structured data.
- The database is configured with appropriate access controls to ensure data security and integrity.

C. Schema Design

To organize the data effectively, multiple schemas are created within the database:

a) Raw Schema (Landing Layer)

- Stores unprocessed data directly ingested from source systems
- Maintains original data format without modifications
- Acts as the initial staging layer

b) Staging Schema

- Contains cleaned and preprocessed data
- Removes duplicates, null values, and inconsistencies
- Prepares data for transformation

c) Analytics Schema (Final Layer)

- Stores fully transformed and structured data
- Includes fact and dimension tables
- Used for reporting, analytics, and visualization

This layered schema approach ensures better data organization, maintainability, and scalability.

D. dbt Configuration

The dbt environment is configured to manage data transformations:

- **Project Initialization:**
  A dbt project is created with folders such as *models, seeds, tests, and macros*
- **Connection Setup:**
  dbt is connected to Snowflake using credentials (account, user, role, warehouse, database, schema)
- **Model Configuration:**
  SQL-based models are defined to transform data from raw to analytics layers
- **Testing Configuration:**
  Built-in and custom tests are defined to ensure data quality
- **Incremental Models:**
  Configured to process only new or updated data, improving performance

---

E. Resource Configuration (Compute & Storage)

a) Snowflake Virtual Warehouse

- Used for executing queries and transformations
- Configured based on workload requirements
- Supports auto-scaling and auto-suspend features to optimize cost

b) Storage Configuration

- Snowflake automatically manages data storage
- Supports scalable and secure storage for large datasets

c) Access Control

- Role-based access control (RBAC) is implemented
- Different roles are assigned for admin, developer, and analyst access

---

F. Environment Configuration

- **Development Environment:** For testing dbt models and queries
- **Production Environment:** For executing finalized pipelines
- Environment variables are used to manage credentials securely

---

G. Integration Setup

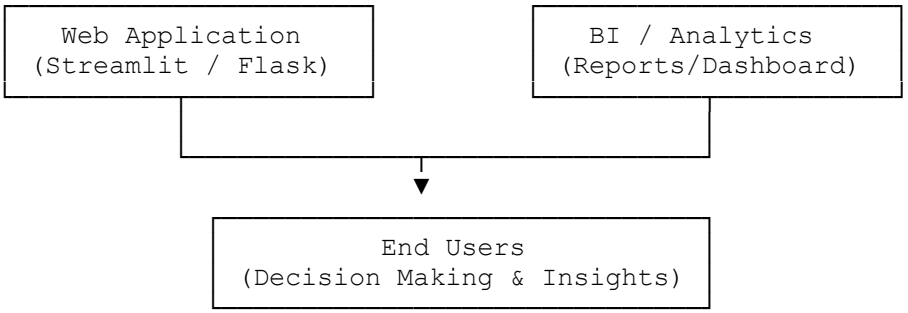
- dbt is integrated with Snowflake for transformation
- Optional integration with Airflow for scheduling
- Web application connects to Snowflake for data visualization

---

Architecture & Flow Diagram

Here is a **clean diagrammatic representation** you can directly use □

```
┌──────────────────────────────────┐
│        Data Sources              │
│   (API / CSV / Database)         │
└──────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────┐
│     Data Ingestion Layer         │
│   (Load into Snowflake)          │
└──────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────┐
│   Snowflake Raw Schema           │
│      (Unprocessed Data)          │
└──────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────┐
│        dbt Transformations       │
│  (Cleaning, Joining, Logic)      │
└──────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────┐
│   Data Modeling Layer            │
│  (Fact & Dimension Tables)       │
└──────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────┐
│  Snowflake Analytics Schema      │
│     (Processed Data)             │
└──────────────────────────────────┘
                 │
┌────────────────┴────────────────┐
▼                                 ▼
```



(Snowflake Raw/Staging and dbt Transformations)

Snowflake (Raw / Staging Layer)

The **Snowflake Raw/Staging Layer** is the initial storage layer in the ETL pipeline where data is ingested in its original or minimally processed form. This layer acts as the foundation of the data pipeline and ensures that the source data is preserved for further processing.

In this layer, data is loaded from various sources such as APIs, CSV files, or databases into Snowflake tables without applying significant transformations. The primary purpose of this layer is to maintain data integrity and provide a reliable starting point for downstream processing.

dbt Transformations

The **dbt (Data Build Tool) Transformation Layer** is the core processing component of the ETL pipeline where raw data is transformed into clean, structured, and analytics-ready datasets.

dbt uses SQL-based models to perform transformations in a modular and reusable manner. These transformations include data cleaning, filtering, joining multiple datasets, applying business logic, and aggregating data. dbt also manages dependencies between models, ensuring that transformations are executed in the correct order.

4 .Development Steps (Data Modeling, Transformations, Incremental Processing, Optimization)

A. Overview of Development Process

The development of the **dbt + Snowflake ETL pipeline** follows a structured approach to ensure efficient data transformation, scalability, and performance. The process includes data modeling, transformation logic implementation, incremental data handling, and performance optimization.

B. Data Modeling

Data modeling is the first step in designing the structure of the data pipeline. It involves organizing raw data into meaningful and structured formats suitable for analysis.

- A **star schema** is typically used, consisting of:
  - **Fact Tables:** Store measurable data such as transactions, sales, or events
  - **Dimension Tables:** Store descriptive attributes such as customer, product, or time
- Relationships between tables are defined using primary and foreign keys
- dbt models are created to represent each table
- Proper data modeling improves query performance and simplifies analysis

C. Data Transformations

Data transformation is the core step where raw data is processed into clean and usable formats using dbt.

- **Cleaning:** Handling missing values, removing duplicates, and correcting inconsistencies
- **Filtering:** Selecting relevant data for analysis
- **Joining:** Combining multiple datasets
- **Aggregation:** Summarizing data (e.g., total sales, counts)
- **Business Logic:** Applying rules specific to the application domain

dbt uses SQL-based models to perform these transformations in a modular and reusable manner.

D. Incremental Processing

Incremental processing is used to improve performance by processing only new or updated data instead of reprocessing the entire dataset.

- dbt supports **incremental models**
- Only newly added or changed records are processed
- Reduces computation time and cost
- Maintains up-to-date data efficiently

Example:
Instead of processing all transactions daily, only new transactions are added to the dataset.

E. Optimization Techniques

Optimization ensures efficient execution of the pipeline and better system performance.

a) Query Optimization

- Writing efficient SQL queries
- Avoiding unnecessary joins and computations

b) Snowflake Optimization

- Using clustering keys for faster queries
- Leveraging Snowflake's auto-scaling features

c) dbt Optimization

- Using incremental models
- Breaking complex transformations into smaller models
- Managing dependencies effectively

d) Resource Optimization

- Configuring appropriate Snowflake warehouse size
- Using auto-suspend and auto-resume features to reduce cost

Diagram: Development Steps

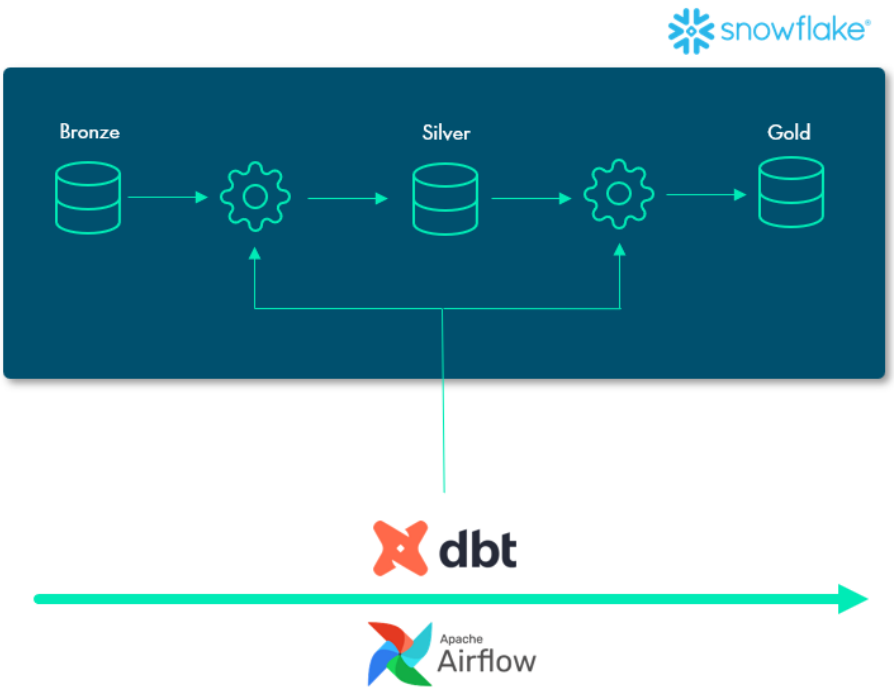
```
┌──────────────────────────────────┐
│        Data Modeling             │
│  (Fact & Dimension Design)       │
└──────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────┐
│      Data Transformation         │
│  (Cleaning, Joins, Logic)        │
└──────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────┐
│   Incremental Processing         │
│  (Process New Data Only)         │
└──────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────┐
│        Optimization              │
│  (Query & Performance Tuning)    │
└──────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────┐
│  Monitoring & Testing            │
│  (Validation & Quality Check)    │
└──────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────┐
│      Final Data Output           │
│    (Analytics Ready Data)        │
└──────────────────────────────────┘
```

---

architecture diagram slides

Slide 1: Title

dbt + Snowflake ETL
System Architecture Diagram



Slide 2: Full Architecture Diagram

Use **Insert → Shapes → Boxes + Arrows** and arrange like this:

```
[Data Sources] ──► [Snowflake Raw]
        │                 │
        ▼                 ▼
 [dbt Transformations] ─► [Warehouse]
                                │
                                ▼
                       [Web App / BI]
```

Slide 3: Layered Architecture

```
Input Layer:
[Data Sources]

Storage Layer:
[Snowflake Raw / Staging]

Processing Layer:
[dbt Transformations]

Output Layer:
[Snowflake Warehouse]
        ↓
[Web Dashboard / BI Tools]
```

Slide 4: Highlight Core Components

```
[Snowflake Raw/Staging]
          │
          ▼
   [dbt Transformations]
```

```
        |
        ▼
[Analytics Data Output]
```

Slide 5: Flow Explanation (Bullet Slide)

- Data is collected from multiple sources
- Stored in Snowflake (raw layer)
- dbt transforms and cleans the data
- Final data stored in warehouse
- Dashboard displays insights

5. Include monitoring, testing, and data quality validation.

Slide: Monitoring

Monitoring in ETL Pipeline

- Continuous tracking of pipeline execution (dbt runs & Snowflake queries)
- Monitoring query performance and resource usage
- Logging of successful and failed jobs
- Real-time alerts for failures or anomalies
- Helps in early issue detection and system stability

Slide: Testing

Testing using dbt

- **Not Null Test:** Ensures no missing values
- **Unique Test:** Prevents duplicate records
- **Relationship Test:** Maintains referential integrity
- **Custom Tests:** Business-specific validations

Slide: Data Quality Validation

Data Quality Checks

- **Accuracy:** Data values are correct
- **Consistency:** Uniform data across tables
- **Completeness:** No missing records
- **Integrity:** Proper relationships maintained

Slide: Integrated Flow (Monitoring + Testing)

```
Data Pipeline
     ↓
Monitoring (Track Performance)
     ↓
Testing (Validate Data)
     ↓
Data Quality Checks
     ↓
Final Reliable Data Output
```

Slide: Key Benefits

- Improved data reliability
- Early error detection
- Reduced data inconsistencies
- Better decision-making support
- High system performance

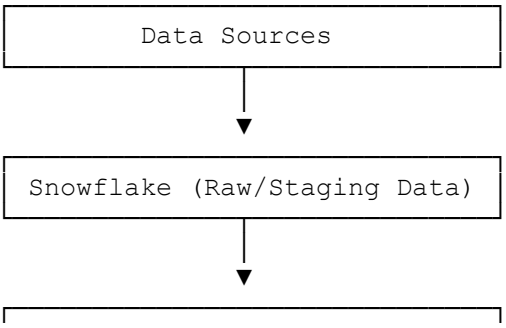
6.Expected Outcomes, Diagrams, and Screenshots

1. Expected Outcomes

The implementation of the **dbt + Snowflake ETL pipeline** is expected to deliver the following outcomes:

- **Fully Automated ETL Pipeline**
  The system automates data extraction, transformation, and loading processes, reducing manual intervention and operational effort.
- **High-Quality, Consistent, and Reliable Data**
  Through systematic data cleaning, testing, and validation, the pipeline ensures that the data is accurate, consistent, and trustworthy.
- **Real-Time or Near Real-Time Data Availability**
  The use of efficient processing and incremental models enables timely data updates for analytics and reporting.
- **Scalable and High-Performance System**
  Leveraging Snowflake's cloud architecture ensures that the system can handle large volumes of data with optimal performance.
- **Interactive Web Dashboard for Visualization**
  The final processed data is presented through a web-based dashboard, enabling users to gain insights using charts, reports, and visual analytics.
- **Improved Decision-Making**
  By providing clean and structured data, the system supports data-driven decision-making for business and analytical purposes.

2. Diagram: Overall Pipeline Output



```
┌──────────────────────────────┐
│      dbt Transformations     │
└──────────────────────────────┘
                │
                ▼
┌──────────────────────────────┐
│   Processed Data Warehouse   │
└──────────────────────────────┘
                │
                ▼
┌──────────────────────────────┐
│  Web Dashboard / BI Tools    │
└──────────────────────────────┘
                │
                ▼
┌──────────────────────────────┐
│          End Users           │
└──────────────────────────────┘
```

---

3. Suggested Screenshots for Report

You should include the following screenshots in your project document:

a) Snowflake Environment

- Database and schema structure
- Raw and processed tables

b) dbt Project Execution

- dbt project folder structure
- Output of `dbt run` and `dbt test`

c) Query Results

- Sample SQL query execution results in Snowflake

d) Web Application Dashboard

- Charts, graphs, and reports
- User interface of the dashboard

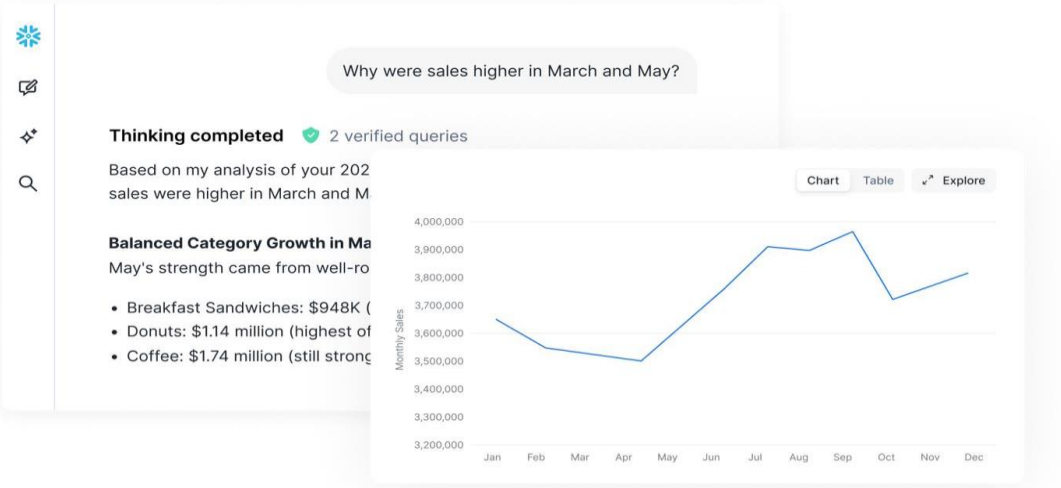
---

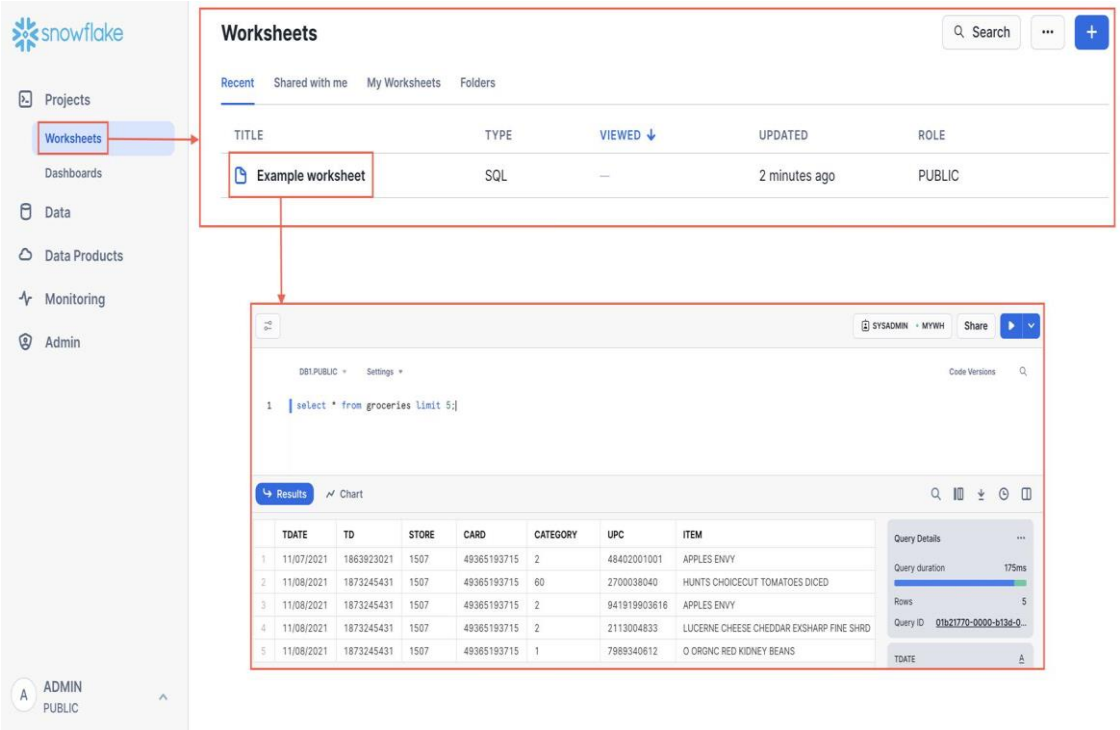
4. Explanation

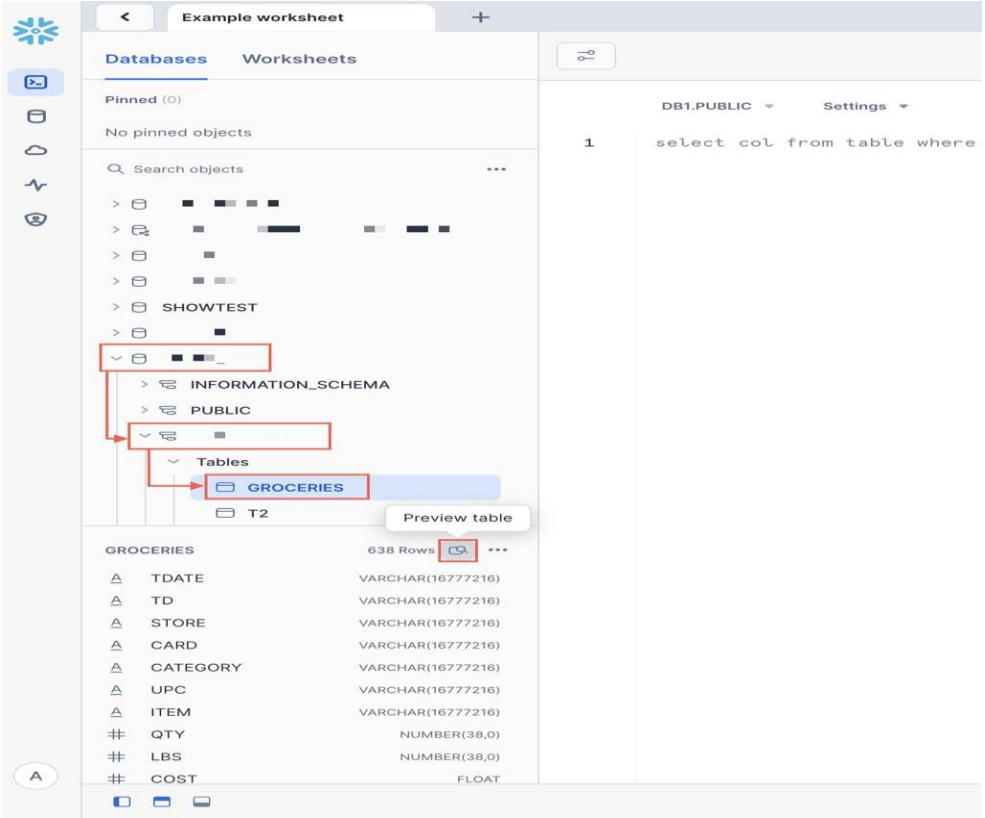
The diagram illustrates the complete flow of data from sources to end users. Data is first stored in Snowflake, then transformed using dbt, and finally delivered through dashboards for analysis. Screenshots provide visual proof of implementation and enhance the credibility of the project documentation.

→Sample Screenshots for Report

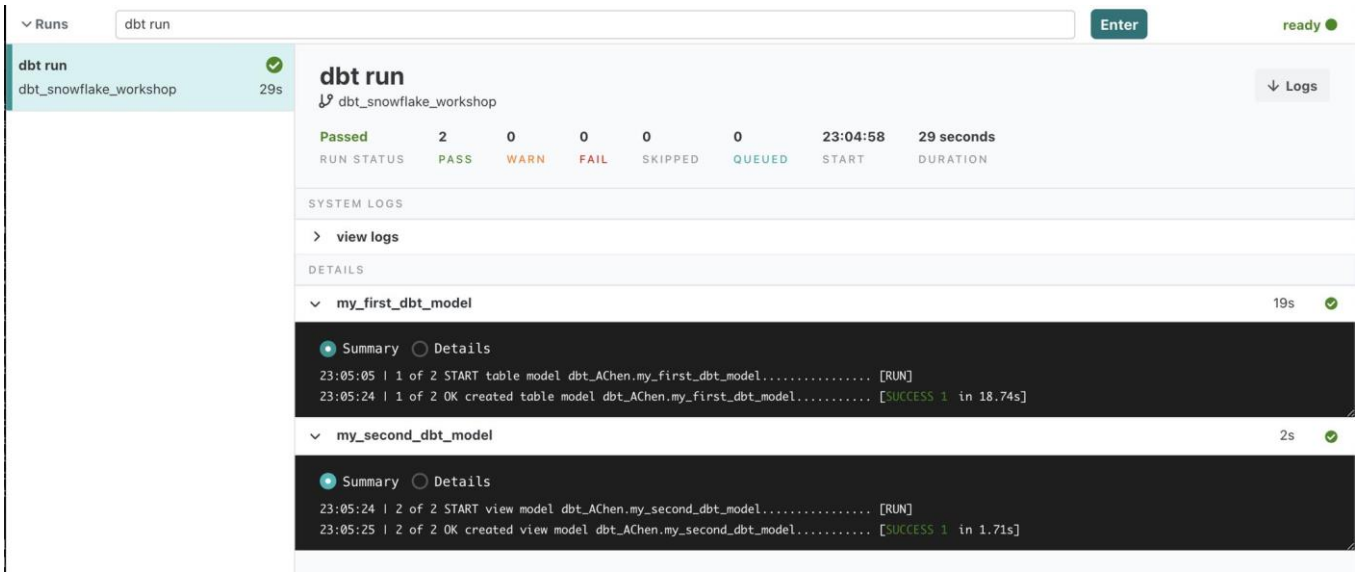
1.Snowflake Dashboard Screenshot



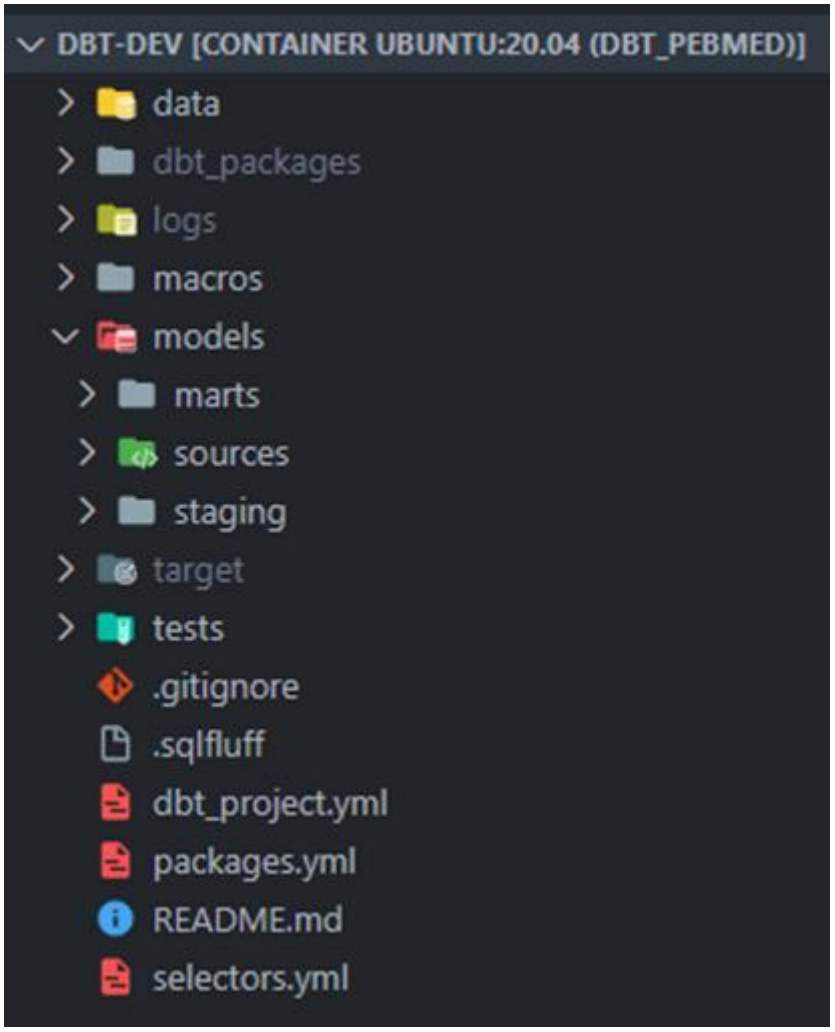




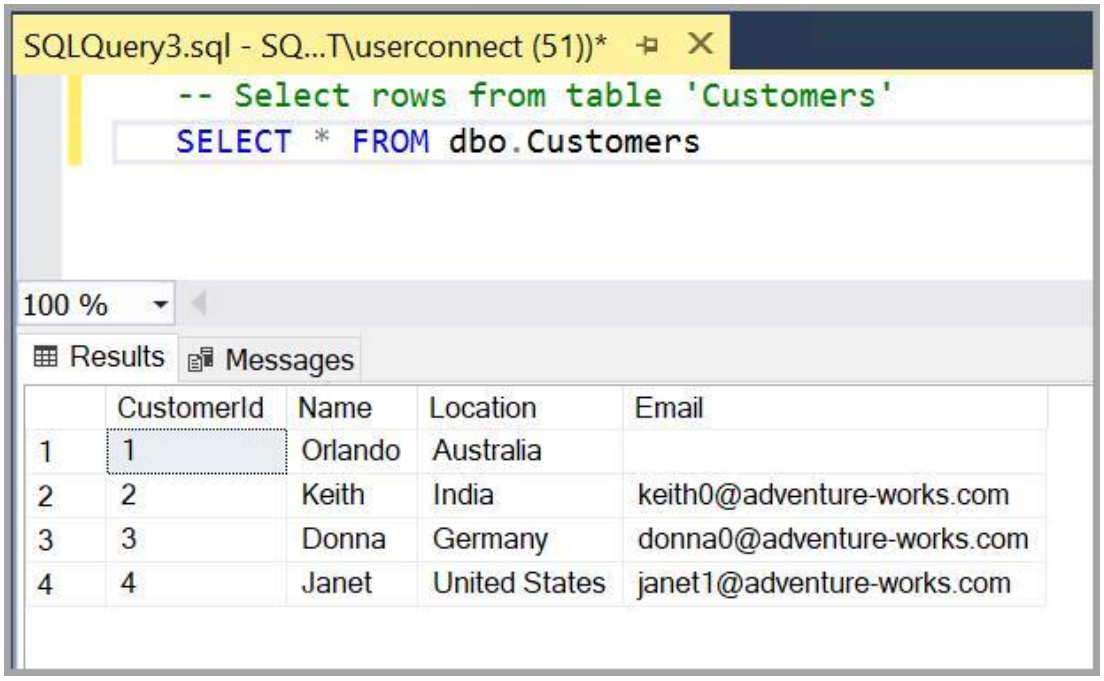
2. dbt Execution Screenshot

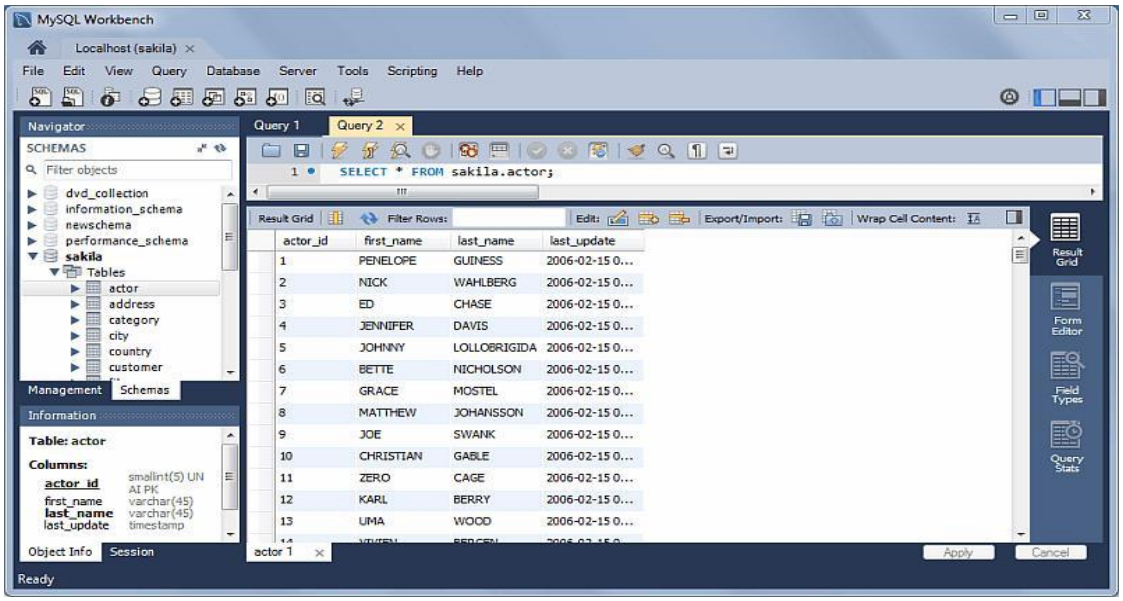


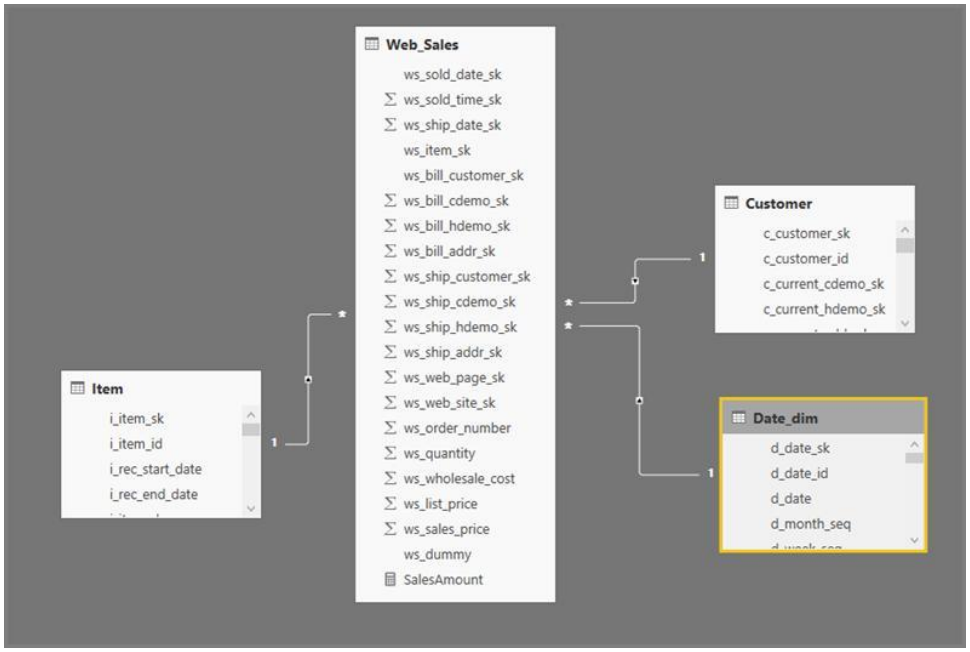
```
PS C:\Users\pvenkat\jaffle_shop> dbt test
23:13:28  Running with dbt=1.4.6
23:13:47  5 of 20 START test not_null_orders_amount ..................................... [RUN]
23:13:48  5 of 20 PASS not_null_orders_amount ..................................... [PASS in 1.17s]
23:13:48  6 of 20 START test not_null_orders_bank_transfer_amount ..................... [RUN]
23:13:49  6 of 20 PASS not_null_orders_bank_transfer_amount ..................... [PASS in 1.21s]
23:13:49  7 of 20 START test not_null_orders_coupon_amount ............................ [RUN]
23:13:51  7 of 20 PASS not_null_orders_coupon_amount ..................................... [PASS in 1.79s]
23:13:51  8 of 20 START test not_null_orders_credit_card_amount ........................ [RUN]
23:13:53  8 of 20 PASS not_null_orders_credit_card_amount ..................................... [PASS in 1.97s]
23:13:53  9 of 20 START test not_null_orders_customer_id ................................ [RUN]
23:13:55  9 of 20 PASS not_null_orders_customer_id ..................................... [PASS in 1.91s]
23:13:55  10 of 20 START test not_null_orders_gift_card_amount ......................... [RUN]
23:13:57  10 of 20 PASS not_null_orders_gift_card_amount ..................................... [PASS in 1.96s]
23:13:57  11 of 20 START test not_null_orders_order_id ................................... [RUN]
23:13:58  11 of 20 PASS not_null_orders_order_id ..................................... [PASS in 1.74s]
23:13:58  12 of 20 START test not_null_stg_customers_customer_id ........................ [RUN]
23:14:00  12 of 20 PASS not_null_stg_customers_customer_id ..................................... [PASS in 1.27s]
23:14:00  13 of 20 START test not_null_stg_orders_order_id ............................... [RUN]
23:14:02  13 of 20 PASS not_null_stg_orders_order_id ..................................... [PASS in 1.92s]
23:14:02  14 of 20 START test not_null_stg_payments_payment_id ........................... [RUN]
23:14:03  14 of 20 PASS not_null_stg_payments_payment_id ..................................... [PASS in 1.93s]
23:14:03  15 of 20 START test relationships_orders_customer_id__customer_id__ref_customers_  [RUN]
23:14:05  15 of 20 PASS relationships_orders_customer_id__customer_id__ref_customers_ .... [PASS in 1.25s]
23:14:05  16 of 20 START test unique_customers_customer_id ............................... [RUN]
23:14:06  16 of 20 PASS unique_customers_customer_id ..................................... [PASS in 1.13s]
23:14:06  17 of 20 START test unique_orders_order_id ..................................... [RUN]
23:14:07  17 of 20 PASS unique_orders_order_id ..................................... [PASS in 1.22s]
23:14:07  18 of 20 START test unique_stg_customers_customer_id ........................ [RUN]
23:14:08  18 of 20 PASS unique_stg_customers_customer_id ..................................... [PASS in 1.29s]
23:14:08  19 of 20 START test unique_stg_orders_order_id ............................... [RUN]
23:14:10  19 of 20 PASS unique_stg_orders_order_id ..................................... [PASS in 1.24s]
23:14:10  20 of 20 START test unique_stg_payments_payment_id ........................... [RUN]
23:14:11  20 of 20 PASS unique_stg_payments_payment_id ..................................... [PASS in 1.18s]
23:14:11
23:14:11  Finished running 20 tests in 0 hours 0 minutes and 42.13 seconds (42.13s).
23:14:11
23:14:11  Completed successfully
23:14:11
23:14:11  Done. PASS=20 WARN=0 ERROR=0 SKIP=0 TOTAL=20
PS C:\Users\pvenkat\jaffle_shop>
```



3. Query Result / Data Output

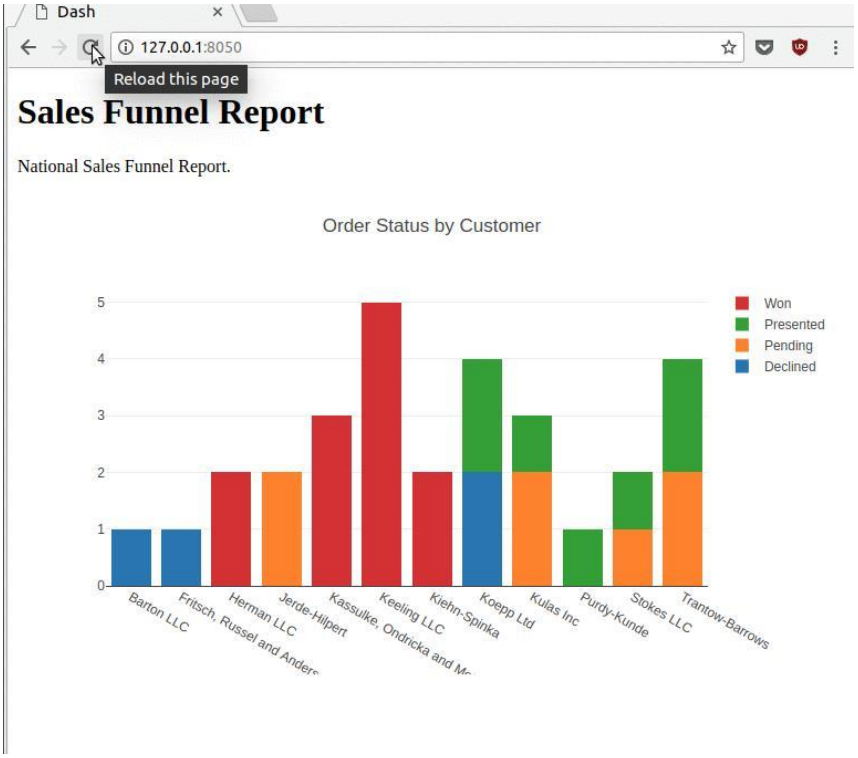






4. Web Dashboard Screenshot

Jaghard
3600%
2%
23%


Dash
127.0.0.1:8050
Reload this page
Sales Funnel Report
National Sales Funnel Report.
Order Status by Customer
Won
Presented
Pending
Declined
0
1
2
3
4
5
Barton LLC
Fritsch, Russel and Anderson
Herman LLC
Jerde-Hilpert
Kassulke, Ondricka and Mertz
Keeling LLC
Kiehn-Spinka
Koepp Ltd
Kulas Inc
Purdy-Kunde
Stokes LLC
Trantow-Barrows